

PKU–Zurich PhD Summer School on Machine Learning for Macroeconomics and Finance

Surrogate models for economics and finance — with the Gaussian-process toolbox
in the appendix

Simon Scheidegger Felix Kübler

HEC Lausanne · University of Zurich

July 6–10, 2026 · Beijing, China

Based on: Chen, Didisheim & Scheidegger (2026), *J. Financial Economics*; Friedl et al. (2023); Kübler, Scheidegger & Surbek (2026, forthcoming in *JPE Macroeconomics*)

https://liboecon.com/2026_summer_school.html

Roadmap of This Lecture

Main part — surrogate models in economics and finance:

- ▶ Why surrogates? From lookup tables to deep neural networks
- ▶ Pseudo-states: model parameters as extra inputs
- ▶ Building, validating, and stress-testing a surrogate
- ▶ Applications: structural estimation, uncertainty quantification, optimal policy, option pricing
- ▶ A pitfall to know: **lazy learning** — and how to fix it

Appendix — the Gaussian-process toolbox (reference material for the afternoon):

- ▶ GP regression and deep kernels
- ▶ Bayesian active learning
- ▶ GPs for dynamic programming; active subspaces (up to 500D)

Hands-on: 01_01_Surrogate_Primer.ipynb (main part); notebooks 01_02–01_09 accompany the appendix.

Learning objectives

By the end of this lecture you should be able to:

- ▶ Explain what a surrogate model is and why economics and finance are ideal terrain for them (we own the data-generating model)
- ▶ Design and train deep surrogates that approximate expensive economic models by treating parameters as pseudo-states
- ▶ Compare approximation methods (polynomials, sparse grids, GPs, DNNs) and recognize when each is appropriate
- ▶ Sketch the main use cases: structural estimation, uncertainty quantification, optimal policy design, and option-pricing calibration
- ▶ Diagnose **lazy learning** in residual-trained surrogates and apply the anchor/distillation and residual-correction fixes

The Gaussian-process machinery behind several of these applications (GP regression, active learning, GP-based dynamic programming) is developed in the **appendix** and in notebooks 01_02–01_09.

Motivation: Why Surrogates?

The computational bottleneck

Economic & financial models are expensive to evaluate:

- ▶ Solving Bellman equations / dynamic programming (DEQNs)
- ▶ PDE-based models (HJB, Black–Scholes for exotics)
- ▶ Monte Carlo simulations for risk measures

Key insight

We know the true model — we can generate *unlimited* training data by solving the model on a design of experiments.

Goal: Build a **surrogate** $\phi(\cdot | \theta_{NN})$ that approximates $f(\cdot)$ but evaluates *orders of magnitude faster*.

Look-up Tables $\rightarrow$ Neural Networks

Traditional: pre-compute & interpolate.

x	$\Phi(x)$
-2.0	0.0228
-1.0	0.1587
0.0	0.5000
1.0	0.8413
2.0	0.9772

Example: standard normal CDF table.

21st-century look-up table:

A neural network memorizes $\mathbf{x} \mapsto y$:

$$\phi(\mathbf{x}_t | \theta_{NN}) \approx y_t = f(\mathbf{x}_t)$$



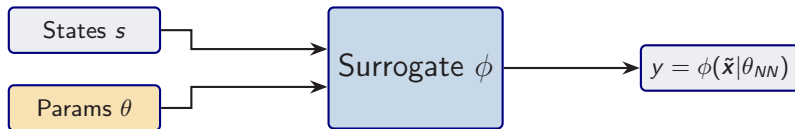
Continuous, differentiable, works in any dimension.

Pseudo-States: The Key Idea

Core insight (Chen, Didisheim & Scheidegger 2026)

Treat model parameters θ as **extra state variables**:

$$\tilde{\mathbf{x}} = \underbrace{(s_1, \dots, s_n)}_{\text{states}}, \underbrace{(\theta_1, \dots, \theta_p)}_{\text{parameters}} \in \mathbb{R}^d, \quad d = n + p.$$



Solve the model **once** over the full augmented space $\Rightarrow$ instant evaluation for *any* parameter configuration.

Building the Surrogate

1. **Design:** Draw $\tilde{\mathbf{x}}_i$ uniformly from $[\underline{x}, \bar{x}]^d$, $i = 1, \dots, m$.
2. **Label:** Compute $y_i = f(\tilde{\mathbf{x}}_i)$ using the expensive model.
3. **Train:** Minimize the empirical risk:

$$\hat{\theta}_{NN} = \arg \min_{\theta_{NN}} \frac{1}{m} \sum_{i=1}^m |\phi(\tilde{\mathbf{x}}_i | \theta_{NN}) - y_i|^2.$$

4. **Validate:** On a held-out test set, compute max / mean absolute error.

Remark

Since we can generate *unlimited* data from the true model, classical overfitting from data scarcity is not the binding constraint. The binding errors come from **approximation** (network capacity vs. true regularity of f) and from **label noise** when f is itself an approximate inner solver (Euler residuals, Monte Carlo simulation).

Why Surrogates $\neq$ Standard ML

Standard supervised learning:

- ▶ Data is *scarce* and noisy
- ▶ Overfitting is a major concern
- ▶ Regularization, early stopping, cross-validation
- ▶ Small learning rates, careful tuning

Surrogate modeling:

- ▶ Data is **unlimited** (we own the model)
- ▶ Labels are **exact** (or noise-free)
- ▶ Can use large epochs, aggressive LR
- ▶ Focus on *approximation error*, not generalization

Once trained

- ▶ **Accurate enough for the downstream task** (relative errors typically 10^{-3} – 10^{-5} on benchmarks; verify per application)
- ▶ **Orders of magnitude faster** than the original model
- ▶ **Differentiable**: gradients via automatic differentiation (backprop)

Speed Comparison: Option Pricing

From Chen, Didisheim & Scheidegger (2026), *J. Financial Economics*:

Task	FFT	Surr. (CPU)	Surr. (GPU)
Single price	23 ms	0.03 ms	—
1-day calib. (6×10^5)	3.3 h	18 s	0.4 s
1-year calib. (1.5×10^8)	≈ 100 d	1.2 h	98 s

Takeaway

Deep surrogates enable **real-time calibration** of option pricing models — tasks that previously took days or were infeasible.

Comparison of Approximation Methods

Criterion	Polynomials/Splines	Sparse Grids	GPs	DNNs
High raw dim. ($d > 10$, no AS)	✗	~	✗	✓
Local features / kinks	✗	✓	~	✓
Irregular domains	✗	✗	✓	✓
Large training set ($N > 10^4$)	✓	~	✗	✓
Built-in uncertainty	✗	✗	✓	✗

- DNNs excel in high dimensions with large data.
- GPs provide **built-in uncertainty quantification** — crucial for active learning and Bayesian inference.
- **Combining both** (GP for moderate d with UQ, DNN for high d) is a powerful strategy.

Application I: Structural Estimation

Goal: Estimate structural parameters θ from data via SMM:

$$\hat{\theta}_{SMM} = \arg \min_{\theta} [m(\theta) - \hat{m}^{\text{data}}]^{\top} W [m(\theta) - \hat{m}^{\text{data}}],$$

where $m(\theta)$ are model-implied moments and computing them naively requires solving and simulating the model for each candidate θ .

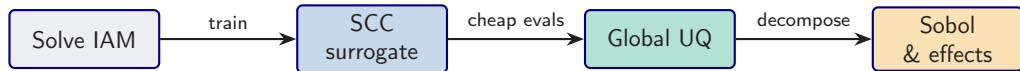
Surrogate-based estimation

1. Train a surrogate policy $\phi(x, \theta)$ over **states and parameters** (pseudo-states).
2. Reuse the trained surrogate to simulate moments $m(\theta)$ without re-solving the model.
3. Evaluate the SMM criterion over many θ values via grid search or standard optimization.

Result: Repeated SMM evaluations in **seconds** instead of weeks.
(Chen, Didisheim & Scheidegger 2026)

Application II: Uncertainty Quantification

DeepUQ (Friedl, Kübler, Scheidegger & Usui 2023):



- ▶ Solve a high-dimensional IAM, train a surrogate for the **social cost of carbon** $\text{SCC}(\theta)$
- ▶ Cheap surrogate evaluations enable **global sensitivity analysis**: Sobol indices, univariate effects
- ▶ Identifies which climate/economic parameters drive SCC uncertainty

Application III: Optimal Policy

Kübler, Scheidegger & Surbek (2026, forthcoming in *JPE Macroeconomics*):

- ▶ High-dimensional integrated assessment model (IAM)
- ▶ GP surrogate + Bayesian Active Learning (BAL) for the value function
- ▶ GP provides *uncertainty quantification* at each point
- ▶ BAL selects training points where uncertainty is highest
- ▶ Result: optimal carbon tax policy

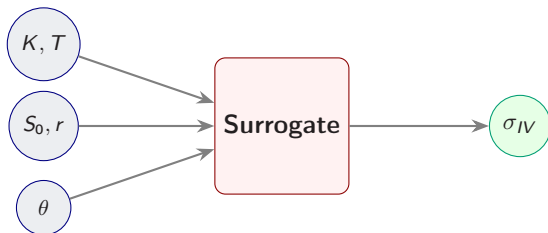
Preview

Parts III and IV of this lecture introduce Gaussian Processes and Bayesian Active Learning — the tools behind this application.

Application IV: The Implied Volatility Surface

Goal: Learn the mapping from option characteristics to implied volatility.

$$\sigma_{IV} = \phi(K, T, S_0, r, \theta_{\text{model}})$$



Benefits.

- ▶ **Calibration:** fit θ by gradient descent on $\sum_i [\phi - \sigma_{IV}^{\text{mkt}}]^2$; $\partial\phi/\partial\theta$ from autograd.
- ▶ **Arbitrage-free:** differentiable penalties for calendar ($\partial_T C \geq 0$) and butterfly ($\partial_K^2 C \geq 0$) violations.
- ▶ **Speed:** real-time pricing for stochastic-vol models without closed form.

Pseudo-States: Policy Parameterization

Split the parameter vector

$\theta = (\theta_{\text{model}}, \theta_{\text{pol}})$: **model** = environment (preferences, technology); **policy** = decision rule (tax path, Taylor coefficients). Both go in as pseudo-state; optimise by autograd through ϕ :

$$\theta_{\text{pol}}^*(\theta_{\text{model}}) = \arg \max_{\theta_{\text{pol}}} \phi(s, \theta_{\text{model}}, \theta_{\text{pol}})$$

Carbon tax

- ▶ Policy: tax trajectory θ_{pol}
- ▶ Model: climate sensitivity θ_{model}
- ▶ Optimal tax for each θ_{model}

Monetary policy

- ▶ Policy: Taylor rule coefficients
- ▶ Model: preference / technology shocks
- ▶ Optimal rule via gradient descent

Key: evaluate welfare for *any* model + policy configuration instantly $\Rightarrow$ no re-solving.

Practical Tips for Surrogate Training

Architecture

- ▶ Swish activation ($x \cdot \sigma(x)$)
- ▶ 3–5 hidden layers, 128–512 neurons
- ▶ Linear output (no activation)

Training

- ▶ $m \geq 10^5$ samples (cheap to generate)
- ▶ Adam, LR 10^{-3} , reduce on plateau
- ▶ Hundreds of epochs (no overfit risk)

Validation

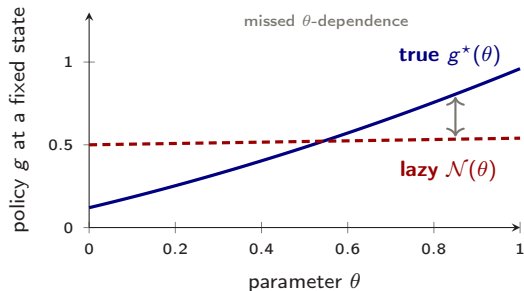
- ▶ Held-out test set
- ▶ Max / mean absolute error, R^2
- ▶ Scatter: prediction vs. truth

Loss functions

- ▶ MSE: $\frac{1}{m} \sum_i (\phi_i - y_i)^2$
- ▶ Relative L^1 : $\frac{1}{m} \sum_i |(\phi_i - y_i)/y_i|$

Pitfall: Lazy Learning — Low Residual, Wrong Policy

- **Symptom.** Lazy training is a **typical pitfall of unsupervised** (residual-only) learning (Chizat, Oyallon & Bach, 2019).
- It drives the training residual down while the policy stays inaccurate across the state–parameter domain.
- Fix a state, and vary a *parameter* θ (a taste, a tax, a shock size).
- The true policy moves with θ ; the lazy surrogate stays *flat*, so it gets the level right on average but the **sensitivity** wrong.



At a fixed state, the true policy varies with θ (blue); the lazy surrogate is almost flat (red dashed), so it misses how the answer should respond to the parameter.

Too flat in the parameter

Low loss, weak parameter dependence: the surrogate is convincing only on average.

The Fix: Anchors and Distillation

- **Fix.** Augment the self-supervised residual loss with a small **semi-supervised distillation** term (Hinton et al., 2015) on a few trusted anchor states:

$$\mathcal{L}^{\text{dist}} = \mathcal{L} + \lambda \sum_{\mathbf{x}_a \in \mathcal{A}} \|\mathcal{N}(\mathbf{x}_a) - p^*(\mathbf{x}_a)\|^2.$$

- $\mathcal{A}$: a handful of anchor states.
- p^* : reference policies from an accurate solver (perturbation, a low-dimensional solve, or a converged reference run).

Remedy

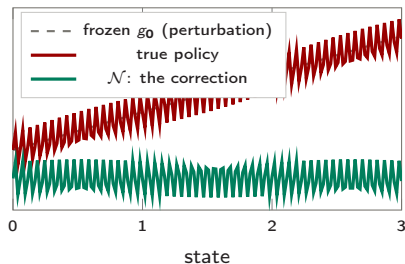
In the deep-UQ application of Friedl et al. (2023) — revisited in Session 3 — only **10–20 anchors** stabilize training, with leave-one-out agreement **within 1%**.

Residual Learning: Freeze and Correct

Idea. Don't ask the network for the *whole* solution. Ask for the **correction** to a cheap baseline:

$$g(\mathbf{x}) = \underbrace{g_0(\mathbf{x})}_{\text{frozen guess}} + \mathcal{N}(\mathbf{x}).$$

- ▶ g_0 : a cheap guess, steady state, perturbation, or certainty equivalent.
- ▶ The net learns only the *small, nonlinear* residual, so **spectral bias bites far less** (He et al., 2016).
- ▶ Sane from iteration zero, natural for PDE / HJB, and it scales: disaster planning in production networks (Carvalho et al., 2025).



The net only fits the small wiggle: frozen guess plus learned correction.

→ **The network is a proofreader, not an author.**

Summary: Deep Surrogate Models

1. **Surrogates** = cheap, accurate replicas of expensive models.
2. **Pseudo-states** (θ as extra inputs) unlock:
 - ▶ Structural estimation (SMM in seconds)
 - ▶ Uncertainty quantification (Sobol indices, univariate effects)
 - ▶ Optimal policy design
3. **Key advantage:** unlimited training data $\Rightarrow$ classical overfitting is not the binding constraint.
4. **Speed:** orders of magnitude faster than original model.
5. **Watch out:** residual-only training can go **lazy** — flat in the parameters despite a low loss; anchor it with a few trusted reference solutions.

Hands-on

Notebook: 01_01_Surrogate_Primer.ipynb — Black–Scholes surrogate with implied volatility inversion.

Questions?

Simon Scheidegger

HEC, University of Lausanne

<https://sischei.github.io>

Materials: [https:](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

[//github.com/sischei/Deep_](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

[Learning_for_Solving_And_](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

[Estimating_Dynamic_Economic_Models](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

Next up:

Session 2:

Structural Estimation with SMM

The **estimate** operator, pointed at today's surrogate.

The following appendix collects the Gaussian-process toolbox used across today's applications.

Appendix

The Gaussian-Process Toolbox

A: GP regression & deep kernels · B: Bayesian active learning · C: GPs for dynamic programming & active subspaces

Why Gaussian Processes?

- ▶ **Nonparametric:** No fixed architecture; the model grows with data.
- ▶ **Built-in uncertainty quantification:** Posterior mean *and* variance at every point.
- ▶ **Principled kernel selection:** Encode prior beliefs about smoothness, periodicity, etc.
- ▶ **Hyperparameter learning:** Marginal likelihood provides a principled objective.

Complementary to DNNs

- ▶ GPs: best for moderate dimensions ($d \leq 10$) with built-in UQ.
- ▶ DNNs: best for high dimensions ($d \gg 10$) with large data.
- ▶ Combining both is a powerful strategy (cf. slide 9).

Foundations: Rasmussen & Williams (2006); MacKay (1992); Krause, Singh & Guestrin (2008). In DP: Engel, Mannor & Meir (2005); Deisenroth, Rasmussen & Peters (2009). Sparse / scalable: Titsias (2009); Hensman, Fusi & Lawrence (2013).

Recall: Multivariate Gaussians

A random vector $\mathbf{x} \in \mathbb{R}^d$ is
multivariate Gaussian if

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$$

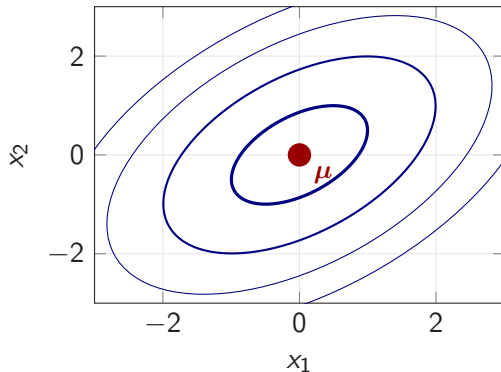
with density

$$p(\mathbf{x}) \propto \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

Key fact

A multivariate Gaussian is **fully characterized** by its mean vector $\boldsymbol{\mu}$ and covariance matrix Σ .

2D Gaussian contours ($\rho = 0.5$)



Joint and Conditional Distributions

Joint distribution:

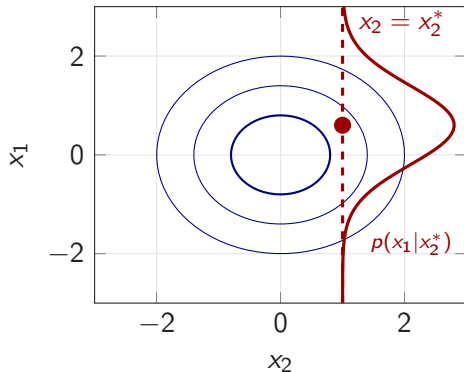
$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}\right)$$

Conditional distribution:

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$$

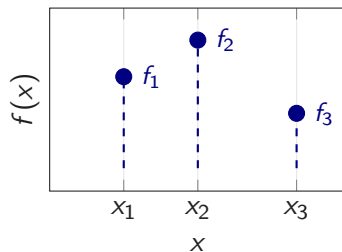
$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

This is the mathematical engine behind
GP regression.



“Slicing” the joint at $x_2 = x_2^*$ yields a 1D Gaussian.

From Observations to Interpolation



Assume function values are jointly Gaussian:

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix}\right)$$

Nearby points (f_1, f_2) should be more correlated than distant ones (f_1, f_3).

Example (SE kernel, $\ell = 2$): $K = \begin{pmatrix} 1.00 & 0.76 & 0.22 \\ 0.76 & 1.00 & 0.61 \\ 0.22 & 0.61 & 1.00 \end{pmatrix}$

Covariance via a **kernel function**: $K_{ij} = k(x_i, x_j)$

$\Rightarrow$ The kernel encodes our prior belief about function smoothness.

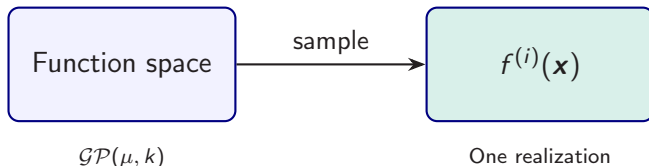
GP: A Distribution Over Functions

Definition

A **Gaussian Process** is a collection of random variables, any finite number of which have a joint Gaussian distribution:

$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')).$$

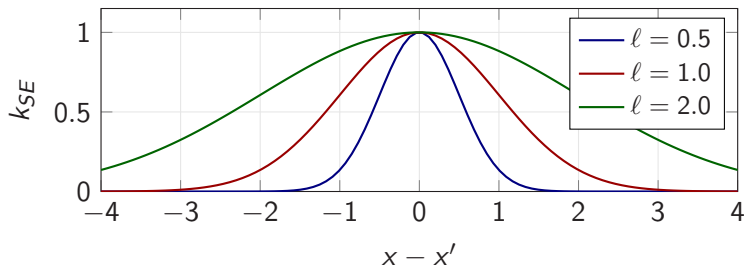
- ▶ **Mean function:** $\mu(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$ (often set to 0).
- ▶ **Covariance function (kernel):** $k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[(f(\mathbf{x}) - \mu(\mathbf{x}))(f(\mathbf{x}') - \mu(\mathbf{x}'))]$.



The Squared Exponential (RBF) Kernel

$$k_{SE}(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

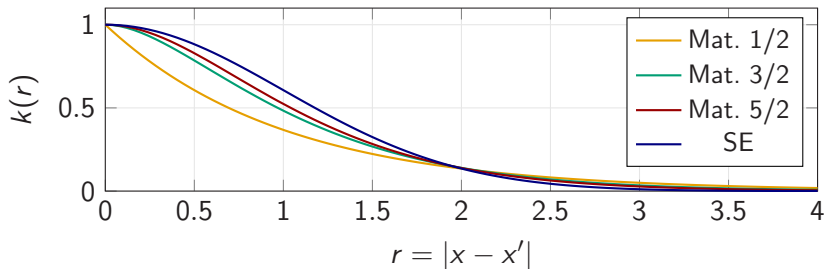
- ▶ ℓ = **length scale** (horizontal correlation range)
- ▶ σ_f^2 = **signal variance** (vertical scale)



Larger $\ell \Rightarrow$ smoother functions; smaller $\ell \Rightarrow$ more wiggly.

The Matérn Kernel Family

$$k_{\text{Mat}}(r) = \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} r}{\ell} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} r}{\ell} \right), \quad r = |x - x'|$$

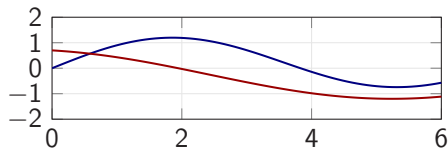


- ▶ ν controls **mean-square smoothness**: $\nu=3/2 \Rightarrow$ once MS-differentiable; $\nu=5/2 \Rightarrow$ twice; SE $\Rightarrow \infty$ -smooth.
- ▶ **Use case**: Matérn when the true function is *not* infinitely smooth (e.g., has kinks).

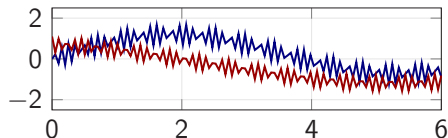
Combining Kernels

Kernels can be **added** and **multiplied** to compose complex priors from simple blocks.

SE kernel (smooth trend)



SE + Periodic (trend + seasonality)



Kernel algebra

- ▶ $k_1 + k_2$ (OR):
high if *either* is high
 $\Rightarrow$ trend + seasonality
- ▶ $k_1 \times k_2$ (AND):
high only if *both* high
 $\Rightarrow$ locally periodic

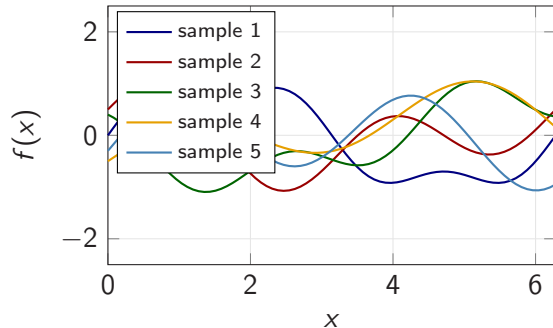
Compose **complex priors** from simple building blocks.

Sampling from the GP Prior

Algorithm

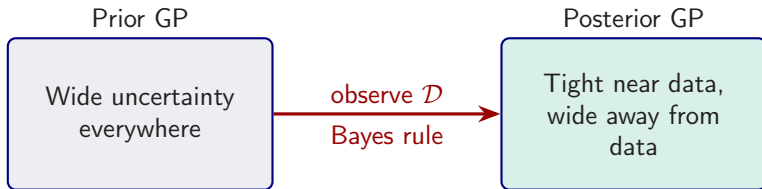
1. Grid $x_{1:N}$ over domain
2. $K_{ij} = k(x_i, x_j)$
3. Cholesky: $K = LL^T$
4. $\mathbf{f}^{(s)} = L \cdot \mathbf{u}$, $\mathbf{u} \sim \mathcal{N}(0, I)$

Five functions drawn from a GP prior with SE kernel ($\ell = 1$, $\sigma_f = 1$).



From Prior to Posterior: Bayes Rule

- ▶ **Training set:** $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$.
- ▶ **Test points:** $\mathbf{X}_*$ where we want predictions.
- ▶ Joint distribution of training outputs $\mathbf{f}$ and test outputs $\mathbf{f}_*$ is Gaussian.



The posterior is *still a GP* — fully characterized by updated mean and covariance.

Noiseless GP Regression

Joint distribution:

$$\begin{pmatrix} \mathbf{f} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} K & K_* \\ K_*^\top & K_{**} \end{pmatrix}\right)$$

where $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$, $(K_*)_{ij} = k(\mathbf{x}_i, \mathbf{x}_*^{(j)})$, $(K_{**})_{ij} = k(\mathbf{x}_*^{(i)}, \mathbf{x}_*^{(j)})$.

Posterior (conditioning on $\mathbf{f}$):

$$\boldsymbol{\mu}_{*|\mathcal{D}} = K_*^\top K^{-1}(\mathbf{f} - \boldsymbol{\mu}) + \boldsymbol{\mu}_* \quad (1)$$

$$\Sigma_{*|\mathcal{D}} = K_{**} - K_*^\top K^{-1} K_* \quad (2)$$

Key insight

- ▶ Near observed data: $K_*^\top K^{-1} K_* \approx K_{**} \Rightarrow$ **small variance**.
- ▶ Far from data: $K_*^\top K^{-1} K_* \approx 0 \Rightarrow$ **large variance** (reverts to prior).

Prediction at a Single Test Point

Noiseless setting ($\sigma_y^2 \rightarrow 0$ limit; GP interpolates training data). For a single test input $\mathbf{x}_*$:

$$\bar{f}_* = \mathbf{k}_*^\top K^{-1} \mathbf{f} = \sum_{i=1}^N \alpha_i k(\mathbf{x}_i, \mathbf{x}_*) \quad (3)$$

$$\sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top K^{-1} \mathbf{k}_* \quad (4)$$

where $\boldsymbol{\alpha} = K^{-1} \mathbf{f}$, $\mathbf{k}_* = (k(\mathbf{x}_1, \mathbf{x}_*), \dots, k(\mathbf{x}_N, \mathbf{x}_*))^\top$. Noisy form ($K \rightarrow K + \sigma_y^2 I$) on next slide.

Interpretation

Posterior mean is a **weighted sum of kernel basis functions** centered at training points:

$$\bar{f}_* = \sum_{i=1}^N \alpha_i k(\mathbf{x}_i, \mathbf{x}_*)$$

Each observation $\mathbf{x}_i$ “pulls” the prediction via its kernel similarity to $\mathbf{x}_*$.

GP Regression with Noisy Observations

Observation model:

$$y = f(\mathbf{x}) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma_y^2)$$

The covariance of *observations* becomes:

$$\text{cov}[y_p, y_q] = k(\mathbf{x}_p, \mathbf{x}_q) + \sigma_y^2 \delta_{pq}$$

In matrix form: replace K with

$$K_y = K + \sigma_y^2 I$$

Effect of noise

- ▶ Predictions no longer interpolate the data exactly.
- ▶ Even at training points, there is **residual uncertainty**.
- ▶ The noise term $\sigma_y^2 I$ also acts as **numerical regularization** (“nugget” / “jitter”).

Noisy GPR: Posterior Equations

Posterior mean and variance with noise:

$$\bar{f}_* = \mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{y} \quad (5)$$

$$\sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{k}_* \quad (6)$$

Noiseless ($\sigma_y^2 = 0$):

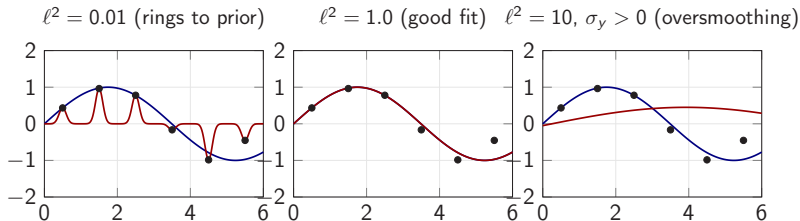
- ▶ Interpolates training points exactly
- ▶ Variance = 0 at training inputs
- ▶ Requires K to be positive definite

Noisy ($\sigma_y^2 > 0$):

- ▶ Smooths through training points
- ▶ Variance > 0 everywhere
- ▶ Better numerical conditioning

In practice: Even for noiseless surrogates, adding a small jitter $\sigma_y^2 \sim 10^{-6}$ improves numerical stability.

The Effect of Kernel Parameters



- ▶ **Blue:** true function $f(x) = \sin(0.9x)$. **Red:** GP posterior mean.
- ▶ **Too small ℓ (noiseless):** interpolates exactly but the posterior rings back to the prior mean between points $\Rightarrow$ poor predictions away from data.
- ▶ **Too large ℓ + noise:** GP smooths through the data, misses high-frequency structure of f .
- ▶ **Bias-variance trade-off in kernel form:** short $\ell \Rightarrow$ low bias but high variance and ringing; long ℓ plus large $\sigma_y^2 \Rightarrow$ high bias and oversmoothing.

$\Rightarrow$ We need a principled way to learn ℓ , σ_f^2 , and σ_y^2 .

Learning Kernel Hyperparameters

Problem: How to choose ℓ , σ_f , σ_y ? $\Rightarrow$ Maximize the **marginal likelihood**.

Marginal likelihood (model evidence, zero-mean prior)

$$\log p(\mathbf{y} | \mathbf{X}) = \underbrace{-\frac{1}{2} \mathbf{y}^\top K_y^{-1} \mathbf{y}}_{\text{data fit}} \underbrace{-\frac{1}{2} \log |K_y|}_{\text{complexity penalty}} \underbrace{-\frac{N}{2} \log(2\pi)}_{\text{constant}}$$

Intuition:

- ▶ **Data fit** $\uparrow$: reward explaining observations
- ▶ **Complexity** $\downarrow$: penalize overly flexible models
- ▶ Balance = **automatic Occam's razor**

Optimization:

- ▶ Gradient ascent on $\log p(\mathbf{y} | \mathbf{X})$
- ▶ Kernel HPs $\theta_{\text{GP}} = \{\ell, \sigma_f, \sigma_y\}$ (separate from structural θ and NN θ_{NN})
- ▶ Closed-form gradients
- ▶ scikit-learn, GPyTorch

Leave-One-Out Predictive Error

Idea. Out-of-sample predictive accuracy without holding out data.

For a GP on $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with noise σ_y^2 :

$$\mu_{-i}(\mathbf{x}_i) - y_i = -\frac{[K_y^{-1}\mathbf{y}]_i}{[K_y^{-1}]_{ii}}, \quad \sigma_{-i}^2(\mathbf{x}_i) = \frac{1}{[K_y^{-1}]_{ii}}.$$

Why free. Once $\text{diag}(K_y^{-1})$ and $K_y^{-1}\mathbf{y}$ are available from the same Cholesky factor used for posterior inference, all n LOO residuals cost only $\mathcal{O}(n)$ extra. No re-training, no held-out split lost.

Use cases. Surrogate-health diagnostic across iterations of GP-VFI (Appendix C) and across active-learning rounds in SMM.

Closed form: Rasmussen & Williams (2006), §5.4.2.

GPR: Numpy Implementation Sketch

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

def kernel(X1, X2, l=1.0, sf=1.0):
    sq = np.sum(X1**2,1).reshape(-1,1) \
        + np.sum(X2**2,1) - 2*X1@X2.T
    return sf**2 * np.exp(-0.5 * sq / l**2)

K = kernel(X_tr, X_tr) + 1e-8*np.eye(N)
L, low = cho_factor(K, lower=True)    # Cholesky
alpha = cho_solve((L, low), y_tr)    # weights

Ks = kernel(X_tr, X_te)
mu   = Ks.T @ alpha                  # mean
v    = cho_solve((L, low), Ks)
var  = kernel(X_te, X_te) - Ks.T @ v # variance
```

See **Algorithm 2.1** in Rasmussen & Williams (2006), and notebook 01_02_GP_and_BAL.ipynb.

GP in Practice: scikit-learn & GPyTorch

scikit-learn:

- ▶ GaussianProcessRegressor
- ▶ Automatic hyperparameter tuning via marginal likelihood
- ▶ Great for prototyping ($N \leq 10^4$)
- ▶ CPU only

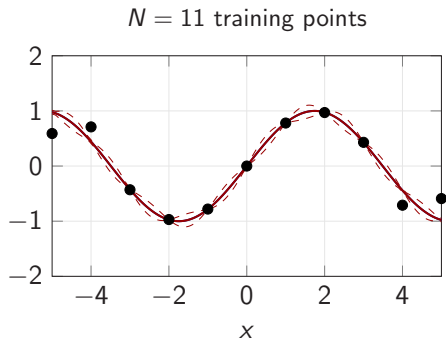
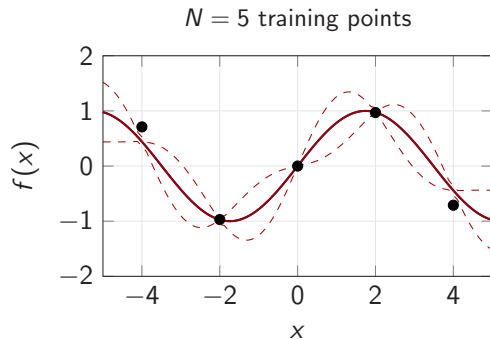
GPyTorch:

- ▶ GPU-accelerated GP inference
- ▶ Scalable to $N > 10^5$ (inducing points, KISS-GP)
- ▶ Integrates with PyTorch ecosystem
- ▶ Production-ready

Hands-on

Notebook: 01_02_GP_and_BAL.ipynb — GP regression from scratch, scikit-learn, and Bayesian Active Learning.

Prediction & Confidence Intervals



Blue: true function. **Red:** posterior mean. **Dashed:** approximate 95% credible band. The band collapses at training inputs (noiseless GP) and widens between them; with more data the band stays uniformly tight.

Algorithmic Complexity

Cost of GP inference

- ▶ **Training:** Cholesky decomposition of $K_y \in \mathbb{R}^{N \times N}$: $O(N^3)$.
- ▶ **Prediction (per test point, after one-off precomputation of $\alpha = K_y^{-1}y$):** $O(N)$ for mean, $O(N^2)$ for variance.
- ▶ **Hyperparameter optimization:** $O(N^3)$ per gradient step.

Scaling strategies for large N

- ▶ **Sparse GPs:** Inducing point methods — approximate K with rank- M matrix ($M \ll N$), cost $O(NM^2)$.
- ▶ **Random features:** Approximate kernel with random Fourier features.
- ▶ **GPyTorch:** KISS-GP, structured kernel interpolation, GPU acceleration.

For the surrogate problems in this course ($N \leq 10^4$, $d \leq 10$), standard GPs are sufficient.

GPs vs DNNs: When to Use Which

Criterion	Gaussian Processes	Deep Neural Networks
Uncertainty quantification	Built-in (posterior variance)	Requires ensembles / MC dropout
Hyperparameter tuning	Marginal likelihood	Grid search / Bayesian opt.
Scaling with N	$O(N^3)$, limit $\sim 10^4$	$O(N)$ per epoch
Scaling with d	Curse of dimensionality	Handles $d \gg 100$
Small data ($N < 100$)	Excellent	Typically underfits
Training	Closed-form	SGD, many epochs
Interpretability	Kernel encodes prior	Black box

Rule of thumb

- ▶ $d \leq 10$, **need UQ**: GPs (+ BAL).
- ▶ $d > 10$, **large data**: DNNs.

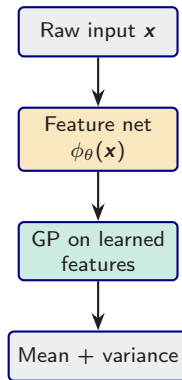
Deep Kernel Learning: Neural Features + GP Uncertainty

Idea: learn a feature map $\phi_{\theta}(\mathbf{x})$ with a neural network, then use

$$k_{\text{DKL}}(\mathbf{x}, \mathbf{x}') = k_{\text{base}}(\phi_{\theta}(\mathbf{x}), \phi_{\theta}(\mathbf{x}'))$$

so the GP operates in the learned feature space.

- ▶ **Front end:** the DNN learns a geometry where the target is smoother.
- ▶ **Back end:** the GP still provides posterior mean and variance.
- ▶ **Training:** optimize the feature map and kernel hyperparameters via the GP marginal likelihood.



When Do Deep Kernels Help?

Standard GP assumption: Euclidean distance in the *raw input space* is informative.

Failure modes:

- ▶ Hidden regimes / thresholds / nonstationarity
- ▶ Inputs are badly warped coordinates of a simple latent state
- ▶ Moderate-to-high-dimensional raw features with strong correlations
- ▶ You still need **uncertainty quantification** or BAL

Rule of thumb

- ▶ If the input is already low-dimensional and smooth: **plain GP** is usually enough.
- ▶ If the latent structure is smooth but the raw geometry is poor: **DKL** is often the right bridge.
- ▶ If d is huge and UQ is not central: **pure DNN surrogate**.

In short: **DKL answers the question “Can we combine DNN feature learning with GP uncertainty?”**

Note on the companion notebook (NB 08). The DKL implementation in NB 08 trains the feature extractor against a hand-crafted teacher feature map for didactic clarity, not jointly with the GP marginal likelihood; see GPyTorch's `DeepKernel` for the joint form.

Summary: Gaussian Process Regression

1. **GPs** = nonparametric Bayesian regression with **built-in UQ**.
2. **Kernel** encodes prior beliefs (smoothness, scale, periodicity).
3. **Posterior**: closed-form mean and variance, conditioned on data.
4. **Hyperparameters**: learned by maximizing the marginal likelihood (automatic Occam's razor).
5. **Limitation**: $O(N^3)$ scaling; best for moderate N and d .
6. **DKL**: neural feature extractor + GP head = useful when raw inputs are warped or moderately high-dimensional, but you still want UQ.

Next: Use the GP's uncertainty to **intelligently select training points** — Bayesian Active Learning.

Motivation: Where to Sample?

Uniform sampling:

- ▶ Places points evenly across the domain
- ▶ Wastes evaluations in smooth regions
- ▶ Cannot adapt to function complexity

Adaptive / active sampling:

- ▶ Concentrates points where the function is **hard to approximate**
- ▶ Uses the model's **uncertainty** to guide selection
- ▶ Fewer evaluations for the same accuracy

Key question

Given a budget of N model evaluations, which input points $\{x_1, \dots, x_N\}$ minimize the overall approximation error?

Bayesian Active Learning: The Idea

Use the GP posterior variance to guide sampling.

A general score function

$$U(\mathbf{x}) = w_{\text{obj}} \cdot \mu(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x})$$

- ▶ $\sigma^2(\mathbf{x})$: posterior variance — **exploration**; sampling where large reduces global uncertainty.
- ▶ $\mu(\mathbf{x})$: posterior mean — **objective-following** bias (useful in Bayesian optimisation, not for global surrogate accuracy).
- ▶ $w_{\text{obj}}, w_{\text{var}} \geq 0$: weights (independent of AR(1) ρ and discount β).

Default: pure exploration ($w_{\text{obj}} = 0$, maximise σ^2 , i.e. uncertainty sampling) for surrogate building. Add the μ -term when also targeting an optimum.

BAL Algorithm

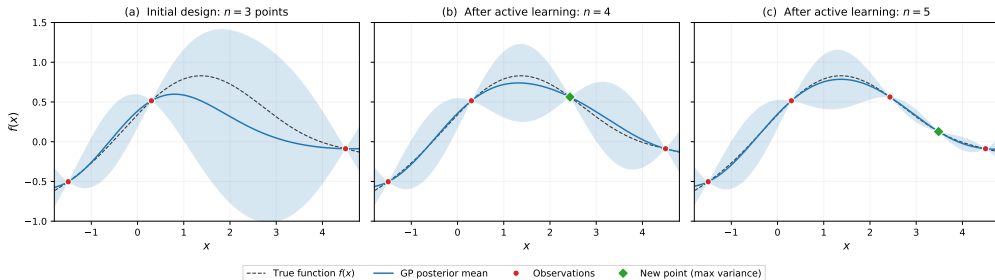
Algorithm 1 Bayesian Active Learning for Surrogates

Require: Initial design $\mathcal{D}_0 = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n_0}$, budget N , candidate set $\mathcal{C}$

- 1: Fit GP on $\mathcal{D}_0$: compute posterior mean $\mu(\cdot)$ and variance $\sigma^2(\cdot)$
 - 2: **for** $t = 1, \dots, N - n_0$ **do**
 - 3: Compute score $U(\mathbf{x}) = w_{\text{obj}} \mu(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{C}$
 - 4: Select $\mathbf{x}_{\text{new}} = \arg \max_{\mathbf{x} \in \mathcal{C}} U(\mathbf{x})$
 - 5: Query the expensive model: $y_{\text{new}} = f(\mathbf{x}_{\text{new}})$
 - 6: Update: $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(\mathbf{x}_{\text{new}}, y_{\text{new}})\}$
 - 7: Refit GP on $\mathcal{D}_t$
 - 8: **end for**
 - 9: **return** Trained GP surrogate
-

BAL in Action: Progressive Refinement

BAL selects each new point where the **posterior variance is largest** (green diamond), causing the credible band to shrink exactly where it was widest.



- (a) 3 initial observations (red dots); GP posterior (blue) deviates in data-sparse regions with wide 95% CI.
- (b) Acquisition function picks max-variance point (green diamond); posterior tightens locally.
- (c) Second iteration fills the remaining gap — 5 points suffice to track the true function.

Key contrast with uniform: BAL concentrates points where the function is hardest to approximate.

BAL in Economics

Grid-free approximation

BAL + GP = grid-free adaptive sparse grids:

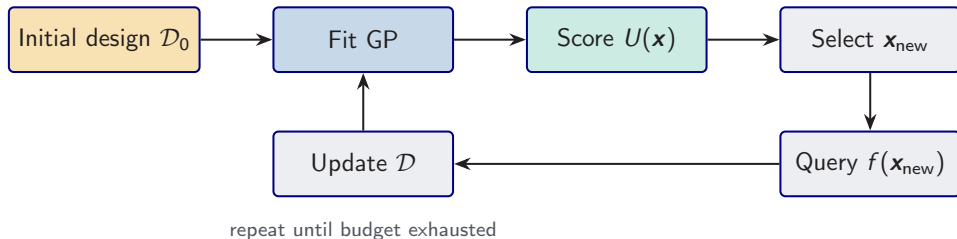
- ▶ No structured grid needed
- ▶ Refinement via posterior variance
- ▶ Scales to $d \leq 10$

Applications

- ▶ **Optimal carbon tax**
- ▶ **Dynamic incentives** (kinks)
- ▶ **Asset pricing** (moderate d)

References: Renner & Scheidegger (2018); Kübler, Scheidegger & Surbek (2026, forthcoming in *JPE Macroeconomics*).

Combining BAL and GP for Surrogates



Why this combination works

- ▶ **GP provides UQ:** posterior variance tells us where the approximation is uncertain.
- ▶ **BAL selects training points:** maximize information gain per model evaluation.
- ▶ **Surrogate = trained GP:** the same GP used for selection serves as the surrogate.

Ideal for $d \leq 10$ when each model evaluation costs minutes to hours.

Summary: Bayesian Active Learning

1. **BAL** = principled data selection using GP uncertainty.
2. **Score function** balances exploration (high σ^2) and exploitation (high μ).
3. **Faster convergence** than uniform sampling, especially for functions with kinks or local features.
4. **Application:** surrogate construction for moderate-dimensional problems.

Hands-on

Notebook: 01_02_GP_and_BAL.ipynb — GP from scratch, scikit-learn GP, and BAL comparison.

Appendix C

Gaussian Processes for Dynamic Programming

Based on: Scheidegger & Bilonis (2019), *J. Comput. Sci.* 33, 68–82.

What is Dynamic Programming?

Dynamic programming (DP) is the core method for solving **sequential decision problems** under uncertainty—the backbone of macro, finance, and structural IO.

The idea: An agent makes decisions over time to maximize total discounted utility:

$$V(\mathbf{x}_0) = \max \sum_{t=0}^{\infty} \beta^t r(\mathbf{x}_t, \xi_t)$$

subject to $\mathbf{x}_{t+1} \sim F(\cdot | \mathbf{x}_t, \xi_t)$.

Bellman's principle of optimality:

$$(TV)(\mathbf{x}) := \max_{\xi} \{ r(\mathbf{x}, \xi) + \beta \mathbb{E}[V(\mathbf{x}_{t+1})] \}$$

T is the **Bellman operator**; the value function V is its fixed point.

Why DP matters:

- ▶ Asset pricing, consumption–savings
- ▶ Growth theory, capital accumulation
- ▶ Labor: job search, human capital
- ▶ Climate: optimal carbon taxation

Challenge: Bellman equation on a **high-dimensional state space**—the curse of dimensionality.

Example: The Stochastic Optimal Growth Model

Workhorse testbed for computational macro (cf. Chapters 2–4). Model with D sectors:

- ▶ State: $\mathbf{k} = (k_1, \dots, k_D)$
- ▶ Controls: consumption $\mathbf{c}$, labor $\mathbf{l}$, investment $\mathbf{i}$
- ▶ Production: $f(k_j, l_j) = A k_j^\psi l_j^{1-\psi}$
- ▶ Shocks: $\epsilon_{t,j} \sim \mathcal{N}(0, \sigma^2)$
- ▶ $k_j^+ = (1 - \delta)k_j + i_j$

D ranges from 1 (textbook) to 500.

Bellman equation:

$$V(\mathbf{k}) = \max_{\mathbf{c}, \mathbf{l}} \{ u(\mathbf{c}, \mathbf{l}) + \beta \mathbb{E}[V(\mathbf{k}^+)] \}$$

Challenge: approximate V on $\mathbb{R}^D$ at each VFI step. Grid methods fail for $D > 20$. **GPs as interpolation** (and the **Active-Subspace GP** — **ASGP** introduced below) $\Rightarrow$ arbitrary geometries, scales to 500D.

Hands-on

01_03_GP_Value_Function_Iteration.ipynb — solves the $D = 1$ case with GP-based VFI.

The Key Idea: GP as Value Function Surrogate

At each VFI step s :

1. Generate n^s training inputs $\mathbf{X}$ in the state space
2. Evaluate Bellman: $t_i^s = (TV^{s-1})(\mathbf{x}^{(i)})$
3. Learn GP surrogate V_{surr} from $\{\mathbf{X}, \mathbf{t}\}$
4. Set $V^s = V_{\text{surr}}$ (posterior mean)
5. Error: $e^s = \|V^s - V^{s-1}\|_\infty / \Delta_V$

GP posterior mean $\rightarrow$ next iterate; GP posterior **variance**
 $\rightarrow$ free uncertainty bounds at every step.

Why GPs instead of grids?

- ▶ Work on **arbitrary geometries**—no tensor-product structure
- ▶ Training points **adaptively chosen**—concentrate near steep gradients
- ▶ Built-in **interpolation uncertainty**
- ▶ Non-converging points can be **discarded**—impossible with grids

Recall

Same GP posterior from Part II, now applied iteratively inside a DP algorithm.

Origin: GPDP — Deisenroth, Rasmussen & Peters (2009, *Neurocomputing*); economics/high-dim extension — Scheidegger & Bilionis (2019, *JoCS*).

Algorithm: ASGP Value Function Iteration

```
1: Input: Initial guess  $V_{\text{next}}$ , tolerance  $\bar{e}$ 
2: while  $e > \bar{e}$  do
3:   Generate  $n^s$  training inputs  $\mathbf{X} = \{\mathbf{x}^{(i)}\} \in [\underline{k}, \bar{k}]^D$ 
4:   for  $\mathbf{x}^{(i)} \in \mathbf{X}$  do
5:     Bellman target:  $t_i = (TV_{\text{next}})(\mathbf{x}^{(i)})$ ;   policy:  $\xi_j(\mathbf{x}^{(i)}) \in \arg \max TV(\mathbf{x}^{(i)})$ 
6:   end for
7:   Learn surrogate  $V_{\text{surr}}$  from  $\{\mathbf{X}, \mathbf{t}\}$  via ASGP (Active-Subspace GP, defined two slides below) or a plain GP for low  $D$ 
8:   Error:  $e = \|V_{\text{surr}} - V_{\text{next}}\|_{\infty}$ ;    $V_{\text{next}} \leftarrow V_{\text{surr}}$ 
9: end while
10: Output:  $V$ , policy functions  $\xi^* = (\xi_1^*, \dots, \xi_{3D}^*)$  (3D controls =  $(c, l, i)$ , one per sector)
```

Key: Training set $\mathbf{X}$ can be **adaptively steered**—non-converging points excluded, new points added. Impossible with grids.

Cheap BAL Inside the VFI Loop

Goal of acquisition. Uniform interpolation accuracy of V , not optimisation $\Rightarrow$ pure exploration.

Acquisition function. Posterior standard deviation only:

$$\mathbf{x}^{\text{next}} \in \arg \max_{\mathbf{x} \in \mathcal{X}^{\text{cand}}} \sigma_{\text{GP}}^s(\mathbf{x}) \quad (\text{no UCB, no EI}).$$

Recipe.

- ▶ Every N VFI iterations, score a Latin-hypercube candidate set
- ▶ Pick top n_{add} by σ_{GP} , with min-spacing constraint to avoid clustering
- ▶ Evaluate Bellman operator at new points (parallel) and append to $\mathcal{D}^s$

Why pure exploration here. A UCB-style acquisition would chase high-value states; we instead want σ_{GP} small **everywhere** so the approximate Bellman operator stays close to a contraction.

Notebook 04 ($N = 12$, $n_{\text{add}} = 2$): active design beats naive LHS on both Bellman residual and LOO-RMSE.

Active Subspaces: Breaking the Curse of Dimensionality

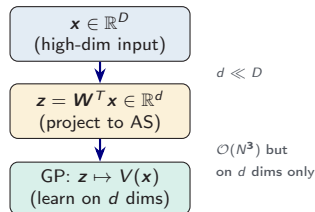
Standard GPs fail for $D \gtrsim 10$ (Euclidean distance uninformative). **Active Subspaces** (AS) find a low-dim manifold of maximal response variation.

Idea: $f(\mathbf{x}) \approx h(\mathbf{W}^T \mathbf{x})$, $\mathbf{W} \in \mathbb{R}^{D \times d}$, $d \ll D$.

Finding $\mathbf{W}$:

1. Collect gradients $\mathbf{g}^{(i)} = \nabla f(\mathbf{x}^{(i)})$
2. $\mathbf{C}_N = \frac{1}{N} \sum \mathbf{g}^{(i)} (\mathbf{g}^{(i)})^T$ (symmetric PSD)
3. Eigendecompose: $\mathbf{C}_N = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T$; find **sharp drop** in λ_i
4. $\mathbf{W} = [\mathbf{v}_1, \dots, \mathbf{v}_d]$ (top d eigenvectors)

The ASGP pipeline:



In the (separable) growth-model benchmark of S&B (2019), the AS dim is $d = 1$ even at $D = 500 \Rightarrow$ tractable. Other models can have larger d ; the spectral-gap test in step 3 decides.

Hands-on: 01_04_Active_Subspace_2D, 01_05..._10D, 01_06..._Nonlinear — AS construction and ASGP vs GP comparison.

When the Active Manifold is Curved

Linear AS assumes the important directions are **linear** combinations of inputs: $f(\mathbf{x}) \approx h(\mathbf{W}^T \mathbf{x})$. If f depends on a **nonlinear aggregate** of its inputs, the linear projection has to carry every direction that enters the aggregate – not the aggregate itself.

Radial-ridge example ($D = 20$):

$$y(\xi) = \exp\left(-[(w_1^T \xi)^2 + (w_2^T \xi)^2]\right).$$

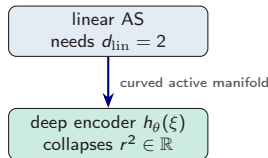
- ▶ $C = \mathbb{E}[\nabla y \nabla y^T]$ has **two** nonzero eigenvalues.
- ▶ Linear AS must pick $d_{\text{lin}} = 2$.
- ▶ Yet the true intrinsic coordinate is the **scalar** $r^2 = (w_1^T \xi)^2 + (w_2^T \xi)^2$ – one dimension.

Symptoms of curvature:

- ▶ Spectral gap ambiguous; λ_i drops slowly.
- ▶ ASGP accuracy plateaus early as d grows.
- ▶ Physics suggests thresholds, ridges, radial coords.

Hands-on: 01_08_Deep_Active_Subspace_Ridge – validates that deep AS recovers $d_{\text{nl}} = 1$ on the radial ridge.

two linear features $\rightarrow$ one nonlinear aggregate



Fix: replace the linear projection $\mathbf{W}^T \mathbf{x}$ by a **learned nonlinear encoder** and jointly train a link function (Tripathy & Bilionis, 2018).

Deep Active Subspaces (Tripathy & Bilonis, 2018)

Parametrize the surrogate as $\hat{f}(\xi) = g(h_\theta(\xi))$, with **encoder** $h_\theta : \mathbb{R}^D \rightarrow \mathbb{R}^d$ and **link** $g_\theta : \mathbb{R}^d \rightarrow \mathbb{R}$ both small MLPs. Setting $h_\theta(\xi) = U_m^\top \xi$ recovers the linear-AS ansatz, so the nonlinear encoder generalizes it.

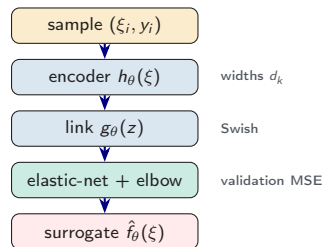
Three design choices:

1. **Widths** decay exponentially $d_k = \lceil D \exp(\eta_w k) \rceil$, $\eta_w = \frac{1}{L} \log(d/D)$.
2. **Swish activation** $\sigma(z) = z/(1 + e^{-z})$ (smooth, non-saturating).
3. **Elastic-net loss**
 $\mathcal{L} = \frac{1}{N} \sum (\hat{f} - y_i)^2 + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2$.

Gradient-free. Learned from (ξ_i, y_i) alone – no ∇f samples, no orthogonality constraint.

Choosing d . No spectral gap; use a **validation-MSE elbow** across $d = 1, 2, 3, \dots$

Rule of thumb: fit linear AS first; escalate to deep AS when (i) no gradients, (ii) ambiguous spectral gap, or (iii) $D \gg 10$ with curved manifold. **Hands-on:** 01_08_...Ridge (curved wins), 01_09_...Borehole (linear wins).



Recipe: Fit a Deep Active Subspace

Self-contained recipe for the Tripathy-Bilonis architecture. All steps rely only on (ξ_i, y_i) pairs – no gradients, no Stiefel constraint, no kernel choice.

Algorithm.

1. **Sample** $(\xi_i, y_i)_{i=1}^N$; 80/20 train/val split.
2. **Standardize** inputs (unit variance), centre outputs.
3. For $d \in \{1, 2, 3, \dots\}$: encoder $h_\theta: \mathbb{R}^D \rightarrow \mathbb{R}^d$ with widths $d_k = \lceil De^{\eta_w k} \rceil$, link $g_\theta: \mathbb{R}^d \rightarrow \mathbb{R}$ (16–32 units); train on $\mathcal{L} = \text{MSE}(\hat{f}, y) + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2$ with Adam + cosine LR.
4. **Pick** d at the *validation-MSE elbow*.
5. **Deploy**: $\hat{f}_\theta$ is the surrogate; $h_\theta(\xi) \in \mathbb{R}^d$ is a nonlinear **active coordinate** usable downstream (GP, sensitivity, optimization).

Sanity checks: (i) validation curve monotone-then-flat in d ; (ii) scatter of $h_\theta(\xi_i)$ vs. the first linear-AS coordinate shows whether the encoder has actually gone *nonlinear*.

Default hyperparameters:

encoder depth L	3
link hidden units	16–32
activation	Swish ($\gamma = 1$)
optimizer	Adam, lr 5×10^{-3}
schedule	cosine, 10^3 – $2 \cdot 10^3$ eps
λ_1, λ_2	$10^{-5}, 10^{-4}$
batch size	full batch if $N \lesssim 10^4$

Samples: $N \approx 50 d_{\text{nl}}$ to find the bottleneck, $\approx 200 d_{\text{nl}}$ for deployment.

Reading the Validation-MSE Elbow

The spectral-gap heuristic of linear AS has no direct analogue for a neural encoder. Tripathy & Billionis (2018) replace it by a **validation-MSE elbow**: train several latent dimensions and stop when extra capacity stops reducing held-out error.

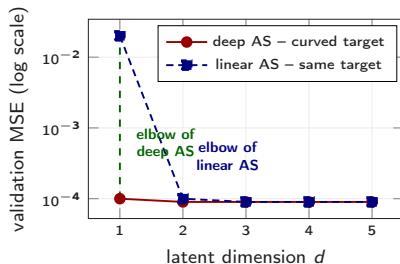
How to read it.

- ▶ Plateau for $d \geq d_{nl} \Rightarrow$ intrinsic dim reached.
- ▶ Linear AS needs $d_{lin} \geq d_{nl}$ whenever the aggregate is nonlinear: the elbow sits **below** the spectral gap.
- ▶ If curves cross (as on the borehole) the function is nearly a ridge and linear AS + polynomial link is more data-efficient.

Three common shapes:

- ▶ **Sharp elbow at $d = 1$** – classic radial / ridge target; deep AS wins.
- ▶ **Gradual decay** – weak low-rank structure; more samples or richer encoder needed.
- ▶ **Flat** – no low-dim structure; return to the original D -dimensional problem.

Operational test: a drop of $< 2\times$ in validation MSE between d and $d + 1$ is **not** worth paying the extra latent dimension for in most applications.



Stylized picture of the radial-ridge experiment (notebook 09): deep AS flattens at $d = 1$, linear AS at $d = 2$.

Parallelization: Embarrassingly Parallel VFI

Bellman evaluations at n^s training points are **independent** $\Rightarrow$ embarrassingly parallel.

MPI workflow:

1. **Broadcast** current V_{next} to all workers
2. **Evaluate:** each worker solves n^s/n_{cpu} Bellman problems
3. **Gather** training targets at master
4. **Fit GP** surrogate (done once, centrally)

Communication cost negligible. Most time in parallelizable Bellman evaluations.

Scaling (stochastic growth model):

D	Nodes	Total CPU h
10	1	0.01 h
50	10	0.5 h
100	20	4 h
200	50	30 h
500	100	~ 3700 h

CPU time grows **sub-exponentially** (< 2 days wall-clock for 500D with 100 nodes).

Results: 1D to 500D Stochastic Growth

Key results (see Figs. 1–5 in the paper):

- ▶ **1D:** error $10^{-1} \rightarrow 10^{-5}$; CI bands negligible with ~ 10 points
- ▶ **10D:** plain GP works; ASGP ($d=1$) equally accurate
- ▶ **50–500D:** ASGP $\sim 10^{-4}$ normalised error

Iteration count is set by the contraction rate of T ($\sim \log(\text{tol}) / \log \beta$), not by D .

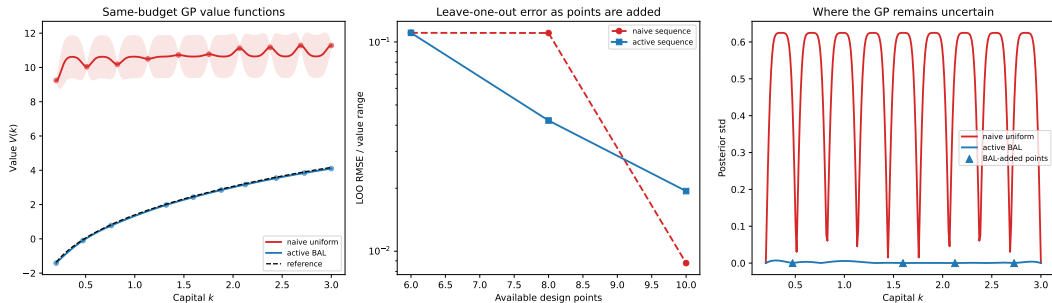
Why this works:

- ▶ Active Subspace dimension is $d = 1$ even for $D = 500 \Rightarrow$ GP operates on 1D projected input
- ▶ Training set adaptively steered (drop non-converging points)
- ▶ Grid methods (Smolyak) cannot handle $D > 100$; ASGP **avoids the explicit grid-based curse** (does not eliminate all high- d costs)

Hands-on

Notebook: 01_03_GP_Value_Function_Iteration.ipynb — solves the 1D growth model with GP-based VFI using scikit-learn. Shows convergence, value function with CI bands, and policy functions.

Active Enrichment vs. Fixed Design: 1D GP-VFI



Same Bellman-label budget (10 points). Active BAL (pure σ_{GP} acquisition, $N = 12$, $n_{\text{add}} = 2$) reduces posterior standard deviation and LOO-RMSE relative to a fixed Latin-hypercube design. Left: value functions; centre: posterior std; right: Bellman residual. (Notebook 04.)

Exercise (5 min)

Consider a 1D growth model with $V(k) = \max_c \{\log(c) + \beta V(k')\}$, $k' = k^{0.36} - c$, $\beta = 0.96$. Suppose you have a GP trained on 3 points: $V(0.5) = -8.0$, $V(1.5) = -2.5$, $V(2.5) = 0.5$.

1. At $k = 1.0$ (a new point not in the training set): the Bellman maximization yields $V^{\text{new}}(1.0) = -4.1$. How does the GP posterior update when you add this observation?
2. Does the GP posterior **variance** at $k = 1.0$ increase or decrease after adding this point?
3. Why is it advantageous that training points can be added or removed freely, unlike grid-based methods?

Solution: One Step of GP-Based VFI

Answer

1. The GP is retrained on 4 points: $\{0.5, 1.0, 1.5, 2.5\}$. The posterior mean $\mu(k)$ now interpolates through all 4 observations exactly (noiseless GP). The posterior mean between existing points adjusts to accommodate the new observation.
2. The posterior variance at $k = 1.0$ **decreases to zero** (the GP passes through the observation exactly). Variance at nearby points also decreases. Variance far from all training points remains large.
3. **Flexibility:** If the Bellman operator at some point fails to converge (e.g., near a constraint), that point can simply be dropped. On a grid, removing a point breaks the tensor-product structure. This is a key advantage of GP-based VFI over grid-based methods.

UQ as a Byproduct & Learning Ergodic Sets

Uncertainty quantification:

The GP posterior variance provides **free UQ** for the interpolated value function at every state point and VFI iteration. Policy uncertainty must be propagated through the maximization step.

Parameter uncertainty propagation:

- ▶ Add uncertain parameters as **pseudo-states**: $\tilde{S} = (\mathbf{k}, \gamma)$ where $\gamma \in [\underline{\gamma}, \bar{\gamma}]$
- ▶ Solve the model once on the extended state space
- ▶ Extract univariate effects: $\mathcal{M}_i(\chi_i) = \mathbb{E}[\mathcal{M}(\Theta) \mid \Theta_i = \chi_i]$
- ▶ Global sensitivity analysis (Sobol-style) in a **single computation**

This avoids the traditional approach of re-solving the model for each parameter value.

Learning ergodic sets:

Many economic models have **irregularly shaped** ergodic sets (ellipsoids, not hypercubes).

1. Solve model on a large domain initially
2. Simulate the economy to generate capital paths $\{\mathbf{k}_t^+\}$
3. Fit a **Bayesian Gaussian Mixture Model** to learn $\hat{\rho}_{\text{ergodic}}(\mathbf{x})$
4. Re-solve on the ergodic set only—sample training points from $\hat{\rho}_{\text{ergodic}}$ instead of the full hypercube

GPs handle this naturally: no grid structure to reconfigure. Training points can be placed **anywhere** in the state space.

GPs vs. DEQNs for Solving Dynamic Models

Both are **complementary tools**—choose based on problem structure:

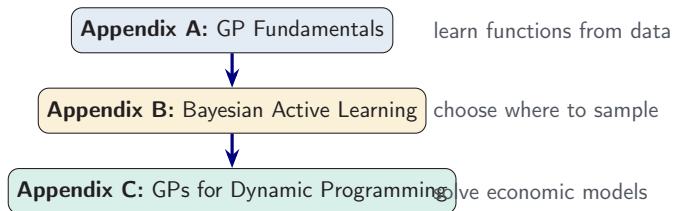
	GP / ASGP	DEQNs (Chapters 2–4)
Method	Value function iteration	Euler equation residuals
Dimensionality	Up to ~ 500 (with AS)	>1000 feasible
UQ built-in	Value (GP variance)	No
Hardware	CPU clusters (MPI)	GPUs
Irregular domains	Yes	Yes
Sensitivity analysis	Cheap with pseudo-states	Pseudo-states or re-training

Use GPs when effective dimension is moderate and you need value-function UQ or sensitivity analysis.

Use DEQNs when D is very large, you have GPU access, or you need complicated market-clearing conditions.

Summary: The Full GP Pipeline for Economics

The same GP machinery, applied at **increasing levels of ambition**:



Reference: Scheidegger & Bilonis (2019), “Machine Learning for High-Dimensional Dynamic Stochastic Economies,” *J. Computational Science* 33, 68–82.

Hands-on

Notebook: 01_03_GP_Value_Function_Iteration.ipynb (in code/01_surrogates_and_gps/).

Recap: The Computational Toolbox

PINNs

- ▶ Continuous-time PDEs
- ▶ Physics loss
- ▶ Mesh-free
- ▶ HJB, Black–Scholes

Deep Surrogates

- ▶ Discrete-time
- ▶ Pseudo-states
- ▶ Fast evaluation
- ▶ Estimation, UQ

GPs + BAL

- ▶ Nonparametric
- ▶ Built-in UQ
- ▶ Active learning
- ▶ Moderate dim

ASGP + VFI

- ▶ GP inside DP
- ▶ Active Subspaces
- ▶ Up to 500D
- ▶ Ergodic sets

All four approaches are **complementary**: choose based on the problem structure, dimensionality, whether UQ is needed, and available hardware.

PINNs are not covered in this school; see the lecture script for a primer.

The Toolbox: When to Use What

Method	Best for	Key strength
DEQN	Discrete-time equilibrium models	Scales to high d , Euler eq. loss
PINN	Continuous-time PDEs/HJB	Mesh-free, physics-constrained
Deep Surrogate	Fast evaluation, estimation, UQ	Pseudo-states, autograd
GP + BAL	Moderate d , need UQ	Built-in uncertainty, active learning
Deep Kernel	Warped inputs + need UQ	Learned features + GP uncertainty

Decision heuristic

1. Is it a continuous-time PDE? $\Rightarrow$ **PINN**.
2. Do you need to re-evaluate for many parameter values? $\Rightarrow$ **Deep Surrogate**.
3. Is $d \leq 10$ and you need UQ / active learning? $\Rightarrow$ **GP + BAL**.
4. Is the latent structure smooth, but the raw input geometry is poor? $\Rightarrow$ **Deep Kernel**.
5. Is it a discrete-time equilibrium model? $\Rightarrow$ **DEQN**.

References & Resources

Key papers:

- ▶ Chen, Didisheim & Scheidegger (2026). *Deep Surrogates for Finance: With an Application to Option Pricing*. J. Financial Econ.
- ▶ Scheidegger & Bilonis (2019). *ML for High-Dim Dynamic Stochastic Economies*. J. Comput. Sci.
- ▶ Kübler, Scheidegger & Surbek (2026). *Using Machine Learning to Compute Constrained Optimal Carbon Tax Rules*. Forthcoming in *JPE Macroeconomics*.
- ▶ Rasmussen & Williams (2006). *GP for ML*. MIT Press.
- ▶ Wilson et al. (2016). *Deep Kernel Learning*. AISTATS.

Notebooks (in `code/01_surrogates_and_gps/`): 01_01_Surrogate_Primer, 01_02_GP_and_BAL, 01_03_GP_Value_Function_Iteration, 01_04-01_06_Active_Subspace (2D, 10D, nonlinear), 01_07_Deep_Kernel_Learning, 01_08_Deep_Active_Subspace_Ridge, 01_09_Deep_AS_vs_Linear_AS_Borehole. The SMM notebooks 02_01 / 02_02 accompany Session 2 (`code/02_structural_estimation_smm/`).

GitHub: [https:](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

[//github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

Questions?